OWNML MACHINE LEARNING CANVAS

Designed for: lstm-cache-eviction-policy | Designed by: Michele Patella | Date: January 21, 2026 | Iteration: 7

PREDICTION TASK

What is the type of task?
Which entity are predictions made on?
What are the possible outcomes to predict?
When are outcomes observed?

TASK TYPE

Sequence prediction (i.e., predict the next access key in a sequence).

PREDICTION ENTITIES

Data keys (i.e., unique identifiers for data items).

PREDICTION OUTCOMES

Logits representing the likelihood of each key of being accessed at each future step.

OUTCOMES OBSERVATION

At the end of the predicted future time horizon.

DECISIONS

How are predictions turned into actionable recommendations or decisions for the end-user? (Mention parameters of the process / application for this.)

FROM PREDICTIONS TO DECISIONS

Model logits and variances (over a predefined time horizon) are combined into normalized eviction scores via confidence-weighted logic (for a given confidence level). These scores are used to rank cache keys, where higher scores indicate lower eviction priority, and keys with the lowest scores—excluding protected ones—are selected for eviction.

VALUE PROPOSITION

Who is the end beneficiary, and what specific pain points are addressed?
How will the ML solution integrate with their workflow, and through which user interfaces?

END BENEFICIARY

Operations/platform teams integrating the solution into a specific system (direct) and its corresponding end-users (indirect).

PAIN POINTS ADDRESSED

1) Ineffective cache eviction strategies
- Rigid, rule-based policies
- Poor suitability for real-world, non-stationary data access patterns
- No use of predictive or probabilistic information
2) Inefficient resource utilization
- Low cache hit rate
- Suboptimal use of fast cache memory
3) High operational and infrastructure costs
- Increased load on backing services
- Higher infrastructure and operational expenses
4) Degraded service performance and User Experience (UX):
- Increased end-user latency
- Lower quality of service
- Negative impact on UX
- Risk of violating Service Level Objectives (SLOs)

ML SOLUTION IN THE WORKFLOW

The solution is integrated via a RESTful API within a microservice-based architecture. The cache management layer provides recent access data information, current cache state, and optional configuration settings, and receives the key(s) to evict when inserting new item(s) into a full cache. The eviction decision is fully programmatic; no user interface is required.

DATA COLLECTION

How is the initial set of entities and outcomes sourced (e.g., database extracts, API pulls, manual labeling)?
What strategies are in place to update data continuously while controlling cost and maintaining freshness?

INITIAL DATA SOURCING

The initial dataset is generated synthetically via programmatic simulation of key access sequences.

CONTINUOS DATA UPDATES

Key access events, already preprocessed by the featurizer for API use, are logged as structured events and appended to the dataset for continuous updates. Only recent access events within a fixed lookback window are used, ensuring fresh data and limiting processing costs.

DATA SOURCES

Where can we get data on entities and observed outcomes? (Mention internal and external database tables or API methods.)

IMPACT SIMULATION

What are the cost/gain values for (in)correct decisions?
Which data is used to simulate pre-deployment impact?
What are the criteria for deployment?
Are there fairness constraints?

MAKING PREDICTIONS

Are predictions made in batch or in real time?
How frequently?
How much time is available for this (including featurization and decisions)?
Which computational resources are used?

PREDICTION TYPE AND FREQUENCY

Real-time and event-driven, triggered whenever the cache requires an eviction decision.

LATENCY CONSTRAINTS

The end-to-end latency is bounded to 3000 ms and is fully asynchronous, occurring outside the user request critical path.

COMPUTATIONAL RESOURCES

Server-side logic and inference using CPU-based resources within a scalable microservice architecture.

BUILDING MODELS

How many models are needed in production?
When should they be updated?
How much time is available for this (including featurization and analysis)?
Which computation resources are used?

FEATURES

What representations are used for entities at prediction time?
What aggregations or transformations are applied to raw data sources?

ENTITY REPRESENTATIONS

Keys are represented by three types of features:
- Temporal: sine and cosine of timestamp (cyclical pattern)
- Local: normalized frequency and recency within lookback window
- Identity: raw key mapped to a learnable embedding

DATA TRANSFORMATIONS AND AGGREGATIONS

Raw data (timestamps and keys) is processed as follows:
1) Remove rows with missing values
2) Encode timestamps trigonometrically, transforming them into sine and cosine components
3) Compute local frequency, representing the normalized count of key occurrences within the lookback window
4) Compute local recency, representing the normalized and reversed distance from key's last occurrences within the lookback window
Preprocessed data is then structured into input sequences of fixed length, providing the model with temporal context for predictions.

MONITORING

Which metrics and KPIs are used to track the ML solution's impact once deployed, both for end-users and for the business?
How often should they be reviewed?